Hamming Code Experiment

Aim:
To implement Hamming Code using Python for detecting and correcting single-bit errors using even parity.

Procedure:
1. Enter the data bits.
2. Calculate the number of required parity bits.
3. Insert parity bits at positions that are powers of 2.
4. Calculate parity values using even parity.
5. Generate the Hamming code.
6. Enter the received Hamming code.
7. Detect and correct any single-bit error.

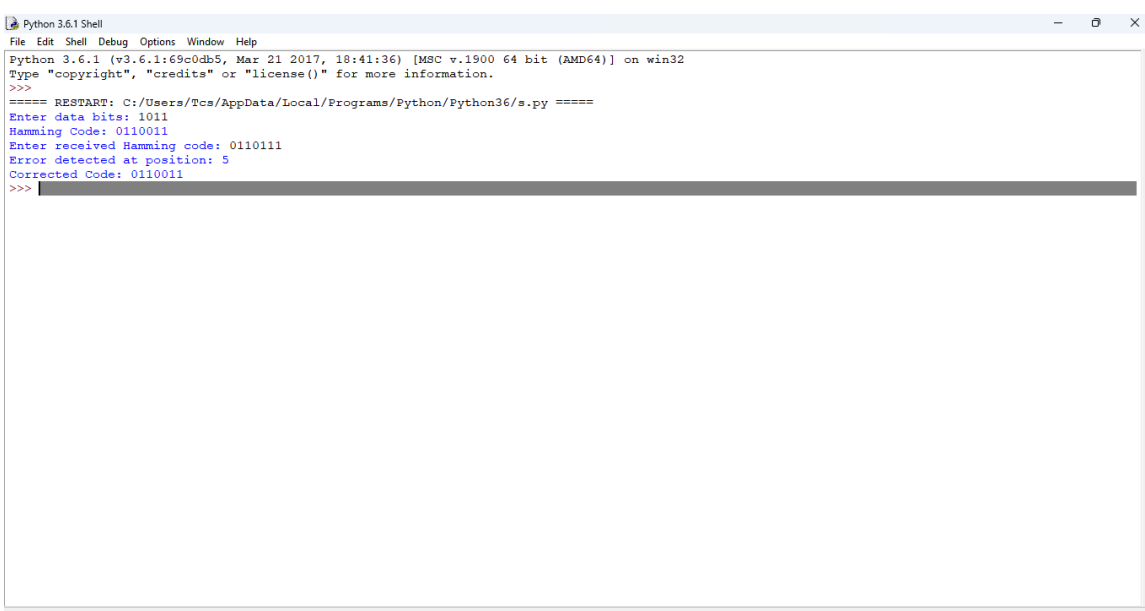
Code:

```
def calculate_parity_bits(m):

    p = 0

    while (2 ** p) < (m + p + 1):

        p += 1

    return p




def insert_parity_bits(data, p):

    n = len(data) + p

    j = 0

    k = 0

    res = []



    for i in range(1, n + 1):

        if i == 2 ** j:

            res.append('0')   # parity bit placeholder
```

```
            j += 1
        else:
            res.append(data[k])
            k += 1

    return ''.join(res)



def calculate_parity(code, p):
    code = list(code)
    n = len(code)

    for i in range(p):
        parity_pos = 2 ** i
        parity = 0

        for j in range(1, n + 1):
            if j & parity_pos:
                parity ^= int(code[j - 1])

        code[parity_pos - 1] = str(parity)

    return ''.join(code)



def detect_and_correct(code, p):
```

```
    n = len(code)
    error_pos = 0

    for i in range(p):
        parity_pos = 2 ** i
        parity = 0

        for j in range(1, n + 1):
            if j & parity_pos:
                parity ^= int(code[j - 1])

        error_pos += parity * parity_pos

    if error_pos == 0:
        print("No error detected.")
        return code
    else:
        print(f"Error detected at position: {error_pos}")
        code = list(code)
        code[error_pos - 1] = '1' if code[error_pos - 1] == '0' else '0'
        return ''.join(code)


def main():
    data = input("Enter data bits: ")
```

```
    m = len(data)
    p = calculate_parity_bits(m)

    code_with_parity = insert_parity_bits(data, p)
    hamming_code = calculate_parity(code_with_parity, p)

    print("Hamming Code:", hamming_code)

    received = input("Enter received Hamming code: ")
    corrected = detect_and_correct(received, p)

    print("Corrected Code:", corrected)


if __name__ == "__main__":
    main()
```



Result:
The Hamming code was generated correctly, and the program successfully detected and corrected a single-bit error in the received data.

Output:
The output displays the generated Hamming code and the corrected Hamming code after error detection.